computational-workflows-project

Davide Bordiga
Marcus Ganske

October 2026

1 Abstract

Reproducible analysis pipelines are a core requirement of FAIR research software. This report presents an RNA-seq workflow built with Nextflow from the nf-core pipeline template and nf-core modules. After quality control (FastQC) and adapter trimming (TrimGalore), reads are processed in two independent branches: genome alignment with HISAT2, followed by duplicate marking (Picard) and transcript quantification with StringTie, and transcript quantification with Salmon, merged into a gene-level TPM table with tximport. All QC outputs are summarised in a single MultiQC report. We describe the nf-core framework, the pipeline architecture, its configuration and outputs, demonstrate the workflow on two samples of a mouse RNA-seq dataset, and assess it against the FAIR principles. Missing features for a complete RNA-seq analysis, including differential expression, additional post-alignment QC and per-sample strandedness, are identified.

2 Introduction

Computational analyses in the life sciences are rarely a single program. They are chains of tools that transform raw data into results, and each tool, version and parameter influences the outcome. Reproducing such an analysis therefore requires more than the code: it needs metadata that describes the input data, the software and its environment [11]. Workflow managers address this problem by describing the steps and their dependencies explicitly, so that the same analysis can be executed on a laptop, a cluster or in the cloud. Nextflow is one such system [5]. It separates the logic of a pipeline from the infrastructure it runs on, and it can run each step in a software container.

The nf-core community builds on Nextflow by providing curated pipelines, a common pipeline template and a repository of shared, tested modules [7]. A pipeline created from the template comes with a parameter schema for input validation, standard configuration profiles, built-in execution reports and a MultiQC summary [6], which makes pipelines from different authors look and behave alike. These features also relate to the FAIR principles for research software: a workflow that is findable, accessible, interoperable and reusable is easier to verify and to build on [19].

RNA sequencing (RNA-seq) is a typical use case. A standard analysis involves quality control, trimming of adapters and low-quality bases, alignment or pseudo-alignment to a reference, and quantification of gene expression, followed by statistical testing [3]. Each of these steps has several established tools, for example FastQC for read quality [1], HISAT2 for spliced alignment [8], StringTie for transcript-level quantification [15] and Salmon for pseudo-alignment based quantification [13].
As a test case we use paired-end bulk RNA-seq data from mice, taken from a study of spared nerve injury (SNI) and sham animals treated with oxycodone or saline [16]. The dataset comprises 16 samples from the brain region. We process two of them, with about 20 million read pairs each, which is large enough to exercise every step of the workflow on a workstation while keeping runtime and disk use manageable.

In this project we built an RNA-seq workflow from the nf-core template and existing nf-core modules instead of writing the processes ourselves. After quality control and trimming, the reads are analysed in two independent branches, a genome alignment branch with duplicate marking and StringTie, and a quantification branch with Salmon, and all quality metrics are combined in one MultiQC report. We run the pipeline on two samples of the dataset, assess it against the FAIR principles, and identify what is missing for a complete RNA-seq analysis. The remainder of this report is organised as follows: Section 3 introduces the FAIR principles, Section 4 the nf-core framework and Section 5 the RNA-seq analysis steps. Sections 6 to 12 describe the pipeline architecture, configuration, parameters, modules and outputs. Section 13 evaluates the pipeline against the FAIR criteria, and Section 14 discusses missing features.

3 FAIR Principles

The FAIR principles state that digital resources such as data, code and workflows should be findable, accessible, interoperable and reusable [18]. They do not prescribe particular technologies. Instead, they describe a range of properties that bring a resource closer to this goal [19]. Although they were formulated for research data, they have since been adapted to research software, which includes analysis pipelines [2].

Findable: To be able to (re)use data they have to be found. Both humans and computers should be able to find data as well as metadata. It is essential for automatic discovery of datasets and services to be machine-readable.

Accessible: When the user has found the data, they need to know how that data can be accessed, including a possible authentication and authorisation.

Interoperable: In most cases the data need to be integrated with other data and it is important that they can interoperate with applications or workflows for analysis, storage, or processing.

Reusable: The last principle and arguably the ultimate goal of FAIR is for the data to be reusable. Data and metadata should be well-described so that they can be replicated and/or combined in different settings, in order to achieve reusability.

4 The nf-core Framework

Workflow management systems let analyses be described as a set of steps with explicit dependencies, and run on laptops, clusters or cloud infrastructure without changing the workflow itself [10]. Nextflow is one such system [5]. nf-core is a community that builds on it: it maintains curated pipelines together with the infrastructure for creating, testing and sharing them [7][1].

Template and tools. New pipelines start from a common template, created with the `nf-core` command-line tools. The template provides the directory layout, default configuration profiles, a parameter schema that validates inputs before a run, execution reports and continuous-integration tests. The same tools lint a pipeline against the community guidelines, install and update modules, and help to build the parameter schema.
This is how pipelines from different authors can be used in the same way.

Modules. Individual tools are packaged as modules in a shared repository. A module typically wraps one tool and declares its inputs and outputs, a pinned container image, a `meta.yml` description and nf-test tests. Tool arguments are not changed in the module: they are set from the pipeline configuration, for example with `ext.args`. A pipeline installs modules at a specific commit, recorded in `modules.json`, includes them in its workflow and connects them through channels. Because lint checks installed modules against the upstream version, modules are used unmodified.

Configuration. Software environments (Docker, Singularity/Apptainer, Conda) and resource settings are selected with profiles and configuration files, so that the same pipeline code runs on different infrastructures.

5 RNA-seq Analysis

RNA sequencing is used in many settings, and no single analysis strategy suits all of them. The choice of steps depends on the organism and on the research question [3]. Most analyses nevertheless share a core: quality control of the raw reads, removal of adapters and low-quality bases, assignment of the reads to genes or transcripts, and quantification of expression. Statistical tests for differential expression usually follow. Expression is commonly reported as transcripts per million (TPM), which corrects read counts for gene length and sequencing depth.
Our pipeline covers the steps up to quantification:

- **Quality control and trimming:** FastQC [1] assesses the raw reads. Trim Galore [9], a wrapper around Cutadapt [12], removes adapters and low-quality bases.
- **Alignment branch:** HISAT2 [8] aligns the reads to the genome in a splice-aware way. SAMtools [4] sorts the alignments, Picard marks duplicate reads for quality control, and StringTie [15] estimates the expression of the annotated genes.

[1] https://nf-co.re

- **Quantification branch:** GffRead [14] builds a transcriptome from the genome and the annotation. Salmon [13] maps the reads to it (pseudo- alignment) and quantifies transcript abundance, and tximport [17] merges the per-sample results into one gene-level TPM table.
- **Reporting:** MultiQC [6] combines the quality metrics of these tools in a single report.

The two branches follow the two common strategies, alignment to the genome and mapping to the transcriptome, and they let us compare the TPM values of the two. Differential expression analysis is not part of the pipeline (Section 14).

6 Pipeline Architecture and Execution

The pipeline follows the nf-core template. Its entry point is `main.nf`, which chains three steps: an initialisation step (`PIPELINE_INITIALISATION`), which validates the parameters and reads the samplesheet, the analysis workflow, and a completion step (`PIPELINE_COMPLETION`). The analysis itself is defined in `workflows/computational-workflows-project.nf`, which includes the modules and connects them through channels (Section 8). Other files configure the run: `nextflow.config` and `conf/` hold parameters and profiles, `nextflow_schema.json` and `assets/schema_input.json` describe the parameters and the samplesheet, and `modules.json` records which version of each nf-core module is installed.
After quality control and trimming (FastQC and TrimGalore), the workflow splits into two independent branches (Figure 1). The alignment branch aligns the reads with HISAT2, sorts the alignments with SAMtools, marks duplicates with Picard, and estimates gene expression with StringTie. The quantification branch builds a transcriptome with GffRead, quantifies it with Salmon, and merges the per-sample results into a gene-level TPM table with tx2gene and tximport. The branches share no intermediate data; both start from the trimmed reads and the same reference files. Their quality metrics, together with the software versions, are combined in one MultiQC report.

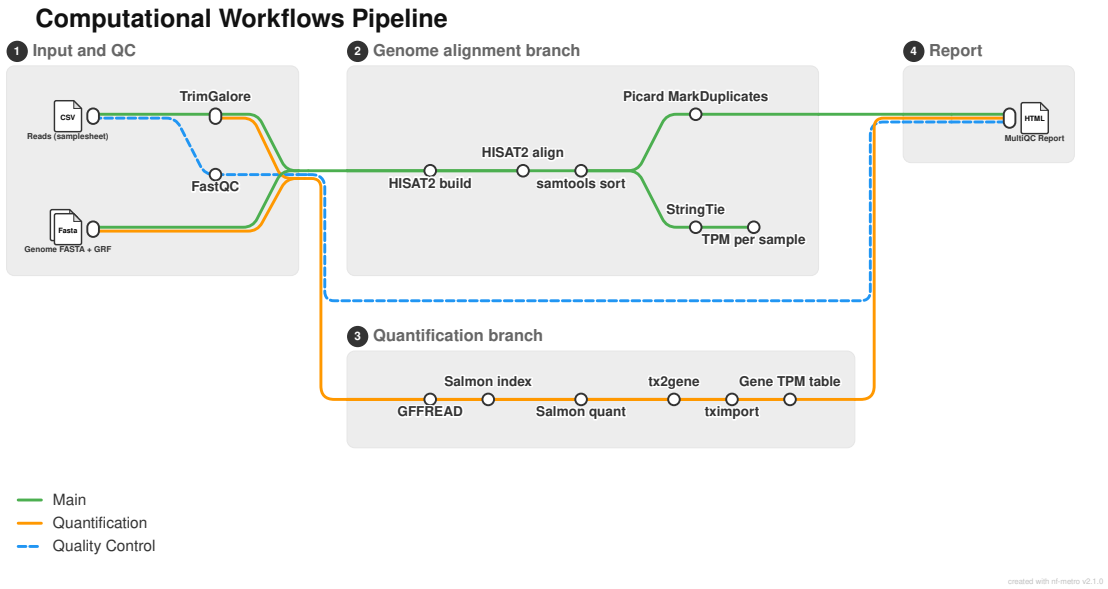


Figure 1: Overview of the pipeline. The alignment route is green, the Salmon route is orange and the quality-control line is dashed blue

The pipeline is run with a container profile and three inputs, a samplesheet and the reference files:

```
nextflow run mganske/computational-workflows-project -r 1.0.0 \
   -profile docker \
   --input samplesheet.csv \
   --fasta <genome.fa> \
   --transcript_fasta <transcript.fa> \
   --gtf <annotation.gtf> \
   --outdir <OUTDIR>
```

7 Configuration and Resources

The pipeline was tested with the Docker profile. The template also defines profiles for Singularity, Apptainer, Podman and Conda, which we did not test. Each module declares its software environment; the images come from Biocontainers (`quay.io/biocontainers`) and Seqera Containers (`community.wave.seqera.io`) and are pinned to version tags. The versions used in a run are written to a `*_software_mqc_versions.yml` file in `pipeline_info/` and appear in the MultiQC report.
Resources are set through the process labels of the template (Table 1); the modules of the pipeline use four of them. We raised the CPU count of the labels to 12 and ran all processes with the `test` profile, whose `resourceLimits` cap the requests at what the workstation offers (16 cores, about 13 GB of RAM). Without such a cap, the larger labels (36 and 72 GB)

cannot be scheduled on this machine. TrimGalore derives its thread count from the allocated CPUs, so its allocation is set separately in `conf/custom.config`.
The pipeline contains no built-in reference. The genome FASTA and the annotation (Ensembl release 102, GRCm38) are passed with `--fasta` and `--gtf`.

Table 1: Process labels used by the pipeline and the resources they request (`conf/base.config`). Requests are multiplied by the attempt number when a task is retried.

Label	CPUs	Memory	Time	Modules
`process_single`	1	6 GB	4 h	MultiQC, tx2gene
`process_low`	12	12 GB	4 h	FastQC, GffRead
`process_medium`	12	36 GB	8 h	TrimGalore, Salmon index, Salmon quant, tximport, StringTie, HISAT2 splice sites, samtools sort, Picard
`process_high`	12	72 GB	16 h	HISAT2 build, HISAT2 align

8 Workflow Graph and Channel Logic

In Nextflow, processes do not call each other. They are connected by *channels*, which carry files and values from the output of one process to the input of the next. A process starts a task as soon as its input channels provide data, so the order of execution follows from the graph in Figure 1, not from the order of the lines in the workflow file. Channels come in two kinds. A *queue channel* holds one item per sample, for example the trimmed reads, and every item creates its own task, which is how samples are processed in parallel. A *value channel* holds a single item that can be read any number of times, which suits the reference files and indices that all samples share.
The graph has one entry (the samplesheet, parsed into a channel of `[meta, reads]` items) and two branches. The trimmed reads from TrimGalore feed both the HISAT2 branch and Salmon, while the reference files feed the index-building steps (HISAT2 build, GffRead, tx2gene). Items carry a `meta` map (sample identifier, `single_end` flag) so that outputs stay linked to their sample.
We used the following operators:

`channel.value` wraps the reference files into value channels, so that every sample can use them.

`map` reshapes items to the form a module expects, for example turning `[meta, file]` into the tuples that Salmon and StringTie require.

`combine` pairs the Salmon index with the annotation and the transcriptome with the genome, producing the input tuples of the modules.

`first` turns the single index bundle into a value channel, so that it is used for *every* sample. Without it, only the first sample is processed.

`collect` gathers the Salmon results of all samples into one list, which tx2gene and tximport need in order to produce a single table.

`mix` merges the outputs of FastQC, HISAT2, Picard and Salmon into the one channel that MultiQC receives.

`channel.topic` reads everything that the modules send to the `versions` topic, without connecting the modules to the consumer.

`distinct` removes identical entries, since a process that ran for every sample reports the same version each time.

`branch` splits the entries into files (modules such as tximport and tx2gene publish a `versions.yml` file) and `[process, tool, version]` tuples (all other modules), which need different processing. This part comes from the template.

`groupTuple` groups the tuples by process name, so that all tools of one process are listed under one heading in the YAML file. This part also comes from the template.

Software versions are not passed between processes explicitly. Each module declares an extra output that is published to the *topic* `versions`: a tuple with the process name, the tool and its version string, which the module obtains by running the tool with `--version`. A topic is a channel that anyone can send to without being connected to a consumer. The workflow reads all values sent to the `versions` topic, removes duplicates, converts them into a YAML file (`pipeline_info/..._software_mqc_versions.yml`) and adds it to the files passed to MultiQC, which shows them in its software versions table. The MultiQC module is the exception. It does not publish its own version to the topic, because it consumes the collected versions and would otherwise wait for its own output. This is also why the nf-core linter reports that MultiQC does not emit its version as a topic.

```
    ch_index_bundle = ch_salmon_index
        .combine(ch_gtf)
        .map { meta, index, gtf -> tuple(meta, index, gtf, []) }

    SALMON_QUANT ( TRIMGALORE.out.reads, ch_index_bundle.first() )
```

9 Pipeline Parameters

Inputs are validated with the nf-schema plugin before a run starts. The parameters are checked against `nextflow_schema.json`, and the samplesheet against `assets/schema_input.json`, which defines its columns, the required fields and a pattern for the FASTQ file names. Invalid input is reported before any process is started. Table 2 lists the parameters that matter for a normal run.

Table 2: Key pipeline parameters (defined in `nextflow.config`).

Parameter	Type	Required	Description
General			
`input`	file (CSV)	yes	Samplesheet with the FASTQ files of each sample; validated by nf-schema
`outdir`	directory	yes	Output directory for all results
`publish_dir_mode`	string	no	How results are published (`copy`, `symlink`, `rellink`)
`validate_params`	boolean	no	Validate the parameters against the schema before the run
Reference data (both branches)			
`fasta`	file (FASTA)	yes	Genome sequence. Used for the HISAT2 index, for the GffRead transcriptome, and as decoy in the Salmon index
`gtf`	file (GTF)	yes	Gene annotation. Used for HISAT2 splice sites, GffRead, tx2gene and StringTie
Alignment branch (HISAT2, StringTie)			
`hisat2_build_memory`	memory	no	Memory threshold. The HISAT2 index is built with splice sites and exons only if the process has at least this much memory
Reporting			
`multiqc_config`	file (YAML)	no	Configuration file for the MultiQC report
Nextflow options (single dash)			
`-profile`	string	yes	Software environment: `docker`, `test`
`-resume`	flag	no	Reuse cached results of a previous run

10 Quality Control Reporting

Every module that produces quality metrics follows the same pattern: the tool reads its input, writes a report file, and the workflow passes that file to MultiQC. FastQC, for example, reads the raw reads and writes `*_fastqc.html` and `*_fastqc.zip` files. The workflow takes the zip file from each sample, adds it to the channel `ch_multiqc_files` with `mix`, and MultiQC aggregates them into one section of its report.

```
1         ch_multiqc_files = ch_multiqc_files.mix(SALMON_QUANT.out.
             results.collect{ it[1] })
```

```
        ch_multiqc_files = ch_multiqc_files.mix(HISAT2_ALIGN.out.
            summary.collect{ it[1] })
        ch_multiqc_files = ch_multiqc_files.mix(
            PICARD_MARKDUPLICATES.out.metrics.collect{ it[1] })
        ch_multiqc_files = ch_multiqc_files.mix(TRIMGALORE.out.log.
            map { _meta, log -> log })
```

All files are collected with `collect` before the call, so that a single MultiQC task receives the files of all samples. MultiQC recognises each tool by the patterns of the file names, not by where the files come from. In addition to the tool reports, the template adds a summary of the workflow parameters, a description of the methods and the software versions. Table 3 shows what reaches MultiQC.

Table 3: Files passed to MultiQC and the report sections they produce.

Source	File passed	Result in the report
FastQC	`*_fastqc.zip`	FastQC section, general statistics
TrimGalore	`*_trimming_report.txt`	passed, but not recognised by MultiQC
HISAT2	`*.hisat2.summary.log`	alignment rate in the general statistics
Picard	`*.markdup.metrics.txt`	duplication rates
Salmon	per-sample result directory	mapping rate in the general statistics
Template	workflow summary, methods description, versions (YAML)	summary, methods and software versions sections

The outputs that are not quality metrics (the TPM tables of tximport and StringTie, the sorted alignments) do not go to MultiQC.

11 Pipeline Modules

Table 4: Modules of the pipeline (all from nf-core/modules).

Module	Tool	Input	Key output
FASTQC	FastQC	raw reads	HTML and zip report
TRIMGALORE	Trim Galore	raw reads	trimmed reads, trimming report
GFFREAD	GffRead	GTF, genome FASTA	transcriptome FASTA
SALMON_INDEX	Salmon	transcriptome, genome (decoy)	Salmon index
SALMON_QUANT	Salmon	trimmed reads, index, GTF	per-sample `quant.sf`
CUSTOM_TX2GENE	Python script	GTF, Salmon results	transcript-to-gene table
TXIMETA_TXIMPORT	tximport	Salmon results, tx2gene	gene and transcript TPM, counts, lengths
HISAT2_EXTRACTSPLICESITES	HISAT2	GTF	splice-site list
HISAT2_BUILD	HISAT2	genome FASTA (+ splice sites)	HISAT2 index
HISAT2_ALIGN	HISAT2	trimmed reads, index, splice sites	BAM, alignment summary
SAMTOOLS_SORT	SAMtools	BAM	sorted BAM and index
PICARD_MARKDUPLICATES	Picard	sorted BAM	duplicate-marked BAM, metrics
STRINGTIE_STRINGTIE	StringTie	sorted BAM, GTF	gene abundance table
MULTIQC	MultiQC	QC files	HTML report

12 Pipeline Outputs

Table 5: Results written to the output directory.

Folder	Files	Content
`fastqc`	`*_fastqc.html`, `*.zip`	read quality reports
`trimgalore`	`*.fq.gz`, `*_trimming_report.txt`	trimmed reads and reports
`salmon`	`quant.sf`, `aux_info/meta_info.json`	per-sample quantification, mapping statistics
`tximport`	`*gene_tpm.tsv`, `*gene_counts.tsv`, `*transcript_tpm.tsv`	merged expression tables (genes × samples)
`hisat2`	`*.bam`, `*.hisat2.summary.log`	genome alignments, alignment summaries
`samtools`	sorted `*.bam`, `*.bai`	sorted and indexed alignments
`picard`	`*.markdup.metrics.txt`	duplication metrics
`stringtie`	abundance table, annotated GTF	gene-level TPM and FPKM per sample
`multiqc`	`multiqc_report.html`, `multiqc_data/`	combined QC report
`pipeline_info`	execution report, timeline, trace, DAG, software versions, parameters	run record

13 Assessment against the FAIR Principles

Findable. The code is in a public GitHub repository with versioned releases (tag [1.0.0]). `nextflow.config` contains metadata (name, authors, description, version).

Accessible. The pipeline is retrieved with `nextflow run` from GitHub and needs no authentication. The tools come from public container registries (Biocontainers, Seqera Containers). The reference data are public Ensembl files release 102 (GRCm38).

Interoperable. Inputs and outputs use standard formats (FASTQ, GTF, BAM/BAI, TSV, YAML, HTML). Parameters and the samplesheet are described by JSON schemas (`nextflow_schema.json`, `assets/schema_input.json`) and validated before a run.

Reusable. The repository has a LICENSE file: the template uses MIT. Documentation consists of the README and `--help`. Versions are pinned by the release tag, by `modules.json`, by container tags and by the minimum Nextflow version. The reference files are not pinned by the pipeline.

Gaps. [see Section 14]

14 Limitations

What's missing for a "strong" RNA-seq pipeline:

1. **Strandedness:** Salmon detects it (`--libType A`, ISR), but StringTie's `--rf` is fixed in the configuration. Nothing passes it to HISAT2.
2. **Post-alignment QC:** no RSeQC or Qualimap (for example gene body coverage, junction saturation).
3. **No differential expression:** the pipeline stops at TPM tables. DESeq2 or edgeR, or the differentialabundance pipeline from week 1, could use the Salmon output. Not abundance.tsv: the matrix is *gene_tpm.tsv.
4. Reference handling: the files are supplied by hand, with no igenomes support, and not pinned by the pipeline.
5. **Testing:** Testing was limited to two of the 16 samples; no automated tests were run.
6. **Resource limits:** the caps live in the test profile, so smaller machines need their own limits (or a patch release).
7. **MultiQC:** TrimGalore's reports aren't recognised, and the HISAT2 index has no splice sites on small machines.
8. **Documentation and CI:** README and –help exist. CI or other Actions do not exist.

Bibliography

[1] Simon Andrews. *FastQC: a quality control tool for high throughput sequence data*. Babraham Bioinformatics. 2010. URL: https://www.bioinformatics.babraham.ac.uk/projects/fastqc/.

[2] Michelle Barker, Neil P. Chue Hong, Daniel S. Katz, et al. "Introducing the FAIR Principles for research software". In: *Scientific Data* 9 (2022), p. 622. DOI: 10.1038/s41597-022-01710-x.

[3] Ana Conesa, Pedro Madrigal, Sonia Tarazona, et al. "A survey of best practices for RNA-seq data analysis". In: *Genome Biology* 17 (2016), p. 13. DOI: 10.1186/s13059-016-0881-8.

[4] Petr Danecek, James K. Bonfield, Jennifer Liddle, et al. "Twelve years of SAMtools and BCFtools". In: *GigaScience* 10.2 (2021), giab008. DOI: 10.1093/gigascience/giab008.

[5] Paolo Di Tommaso et al. "Nextflow enables reproducible computational workflows". In: *Nature Biotechnology* 35 (2017), pp. 316–319. DOI: 10.1038/nbt.3820.

[6] Philip Ewels et al. "MultiQC: summarize analysis results for multiple tools and samples in a single report". In: *Bioinformatics* 32.19 (2016), pp. 3047–3048. DOI: 10.1093/bioinformatics/btw354.

[7] Philip A. Ewels et al. “The nf-core framework for community-curated bioinformatics pipelines”. In: *Nature Biotechnology* 38 (2020), pp. 276–278. DOI: 10.1038/s41587-020-0439-x.

[8] Daehwan Kim et al. “Graph-based genome alignment and genotyping with HISAT2 and HISAT-genotype”. In: *Nature Biotechnology* 37 (2019), pp. 907–915. DOI: 10.1038/s41587-019-0201-4.

[9] Felix Krueger. *Trim Galore: a wrapper around Cutadapt and FastQC*. Babraham Bioinformatics. URL: https://www.bioinformatics.babraham.ac.uk/projects/trim_galore/.

[10] Jeremy Leipzig. “A review of bioinformatic pipeline frameworks”. In: *Briefings in Bioinformatics* 18.3 (2017), pp. 530–536. DOI: 10.1093/bib/bbw020.

[11] Jeremy Leipzig et al. “The role of metadata in reproducible computational research”. In: *Patterns* 2.9 (2021), p. 100322. DOI: 10.1016/j.patter.2021.100322.

[12] Marcel Martin. “Cutadapt removes adapter sequences from high-throughput sequencing reads”. In: *EMBnet.journal* 17.1 (2011), pp. 10–12. DOI: 10.14806/ej.17.1.200.

[13] Rob Patro et al. “Salmon provides fast and bias-aware quantification of transcript expression”. In: *Nature Methods* 14 (2017), pp. 417–419. DOI: 10.1038/nmeth.4197.

[14] Geo Pertea and Mihaela Pertea. “GFF Utilities: GffRead and GffCompare”. In: *F1000Research* 9 (2020), p. 304. DOI: 10.12688/f1000research.23297.2.

[15] Mihaela Pertea et al. “Transcript-level expression analysis of RNA-seq experiments with HISAT, StringTie and Ballgown”. In: *Nature Protocols* 11 (2016), pp. 1650–1667. DOI: 10.1038/nprot.2016.095.

[16] Kerri D. Pryce et al. “Oxycodone withdrawal induces HDAC1/HDAC2-dependent transcriptional maladaptations in the reward pathway in a mouse model of peripheral nerve injury”. In: *Nature Neuroscience* 26.7 (2023), 1229–1244. DOI: 10.1038/s41593-023-01350-3.

[17] Charlotte Soneson, Michael I. Love, and Mark D. Robinson. “Differential analyses for RNA-seq: transcript-level estimates improve gene-level inferences”. In: *F1000Research* 4 (2015), p. 1521. DOI: 10.12688/f1000research.7563.2.

[18] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, et al. “The FAIR Guiding Principles for scientific data management and stewardship”. In: *Scientific Data* 3 (2016), p. 160018. DOI: 10.1038/sdata.2016.18.

[19] Mark D. Wilkinson et al. “A design framework and exemplar metrics for FAIRness”. In: *Scientific Data* 5 (2018), p. 180118. DOI: 10.1038/sdata.2018.118.